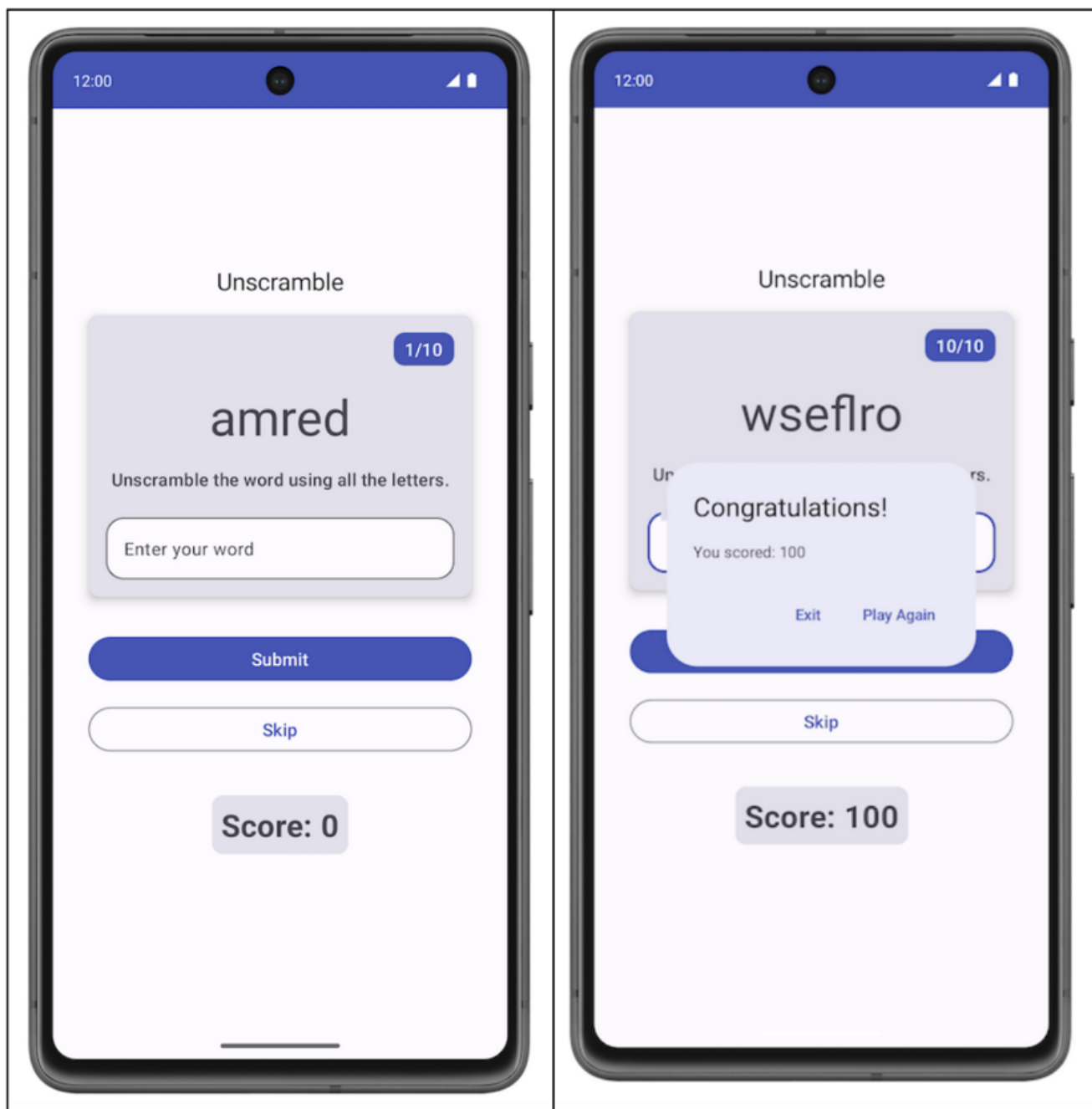
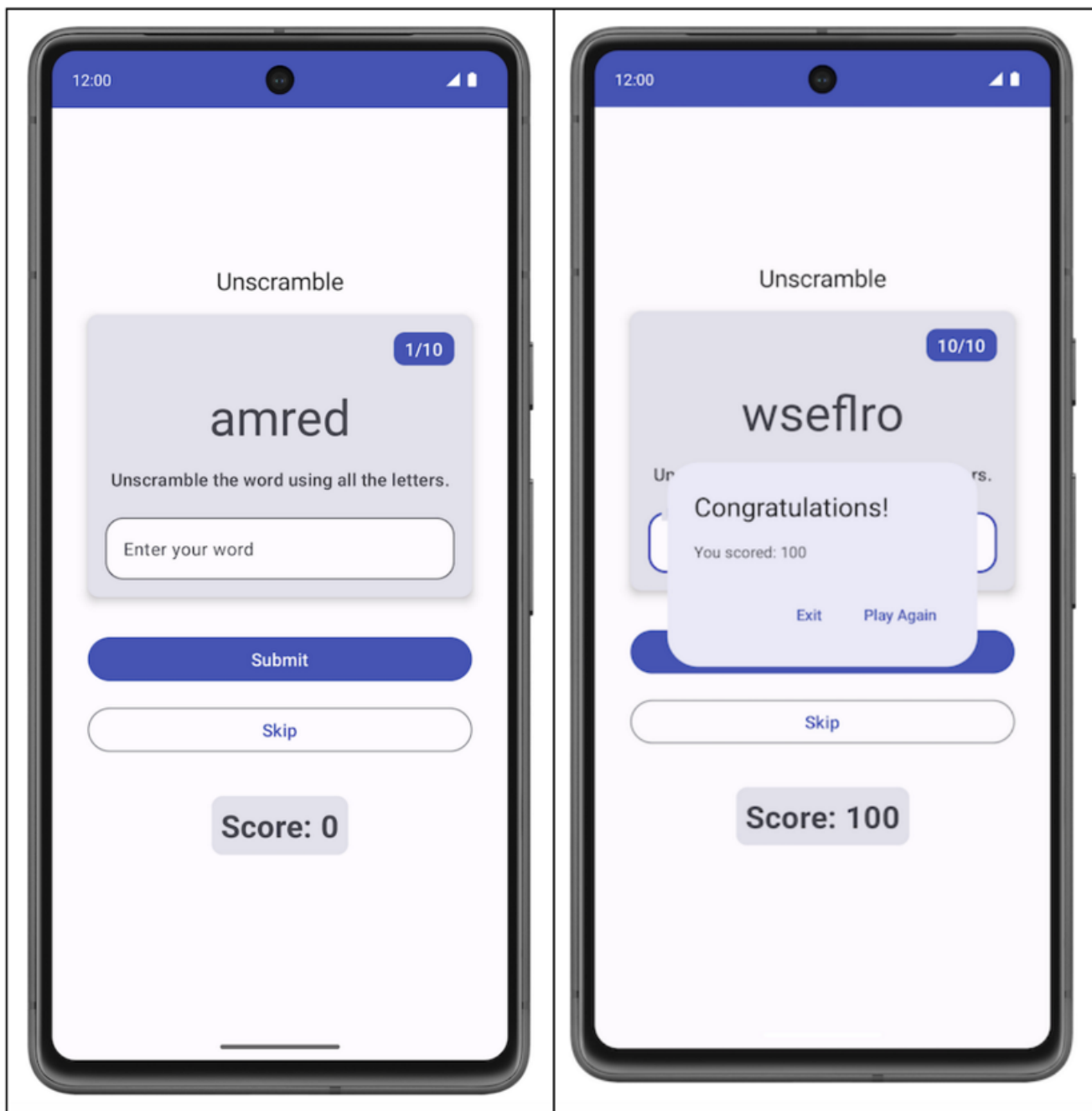


Практическая работа 27. Написание модульных тестов для ViewModel

Этот урок научит вас писать модульные тесты для проверки компонента `ViewModel`. Вы добавите модульные тесты для игрового приложения `Unscramble`. Приложение `Unscramble` - это увлекательная игра в слова, в которой пользователи должны угадать зашифрованное слово и заработать очки за правильное угадывание. На следующем изображении показан предварительный просмотр приложения:





В коделабе «Написание автоматизированных тестов» вы узнали, что такое автоматизированные тесты и почему они важны. Вы также узнали, как реализовать модульные тесты.

Вы узнали:

Автоматизированное тестирование - это код, который проверяет правильность работы другого кода.

Тестирование - важная часть процесса разработки приложений. Постоянно запуская тесты для своего приложения, вы можете проверить его функциональное поведение и удобство использования до того, как выпустите его публично. С помощью модульных тестов можно тестировать функции, классы и свойства. Локальные модульные тесты выполняются на рабочей станции, то есть в среде разработки без использования устройства Android или эмулятора. Другими словами, локальные тесты выполняются на вашем компьютере. Прежде чем продолжить, убедитесь, что вы завершили работу над кодовыми лабораториями Write automated tests и ViewModel and State in Compose.

Необходимые условия

- Знание языка Kotlin, включая функции, лямбды и композиты без состояния
- Базовые знания о том, как создавать макеты в Jetpack Compose
- Базовые знания о Material Design
- Базовые знания о том, как реализовать ViewModel

Что вы узнаете

- Как добавить зависимости для модульных тестов в файл `build.gradle.kts` модуля приложения
- Как создать стратегию тестирования для реализации модульных тестов
- Как писать модульные тесты с помощью JUnit4 и понимать жизненный цикл тестового экземпляра
- Как выполнять, анализировать и улучшать покрытие кода

Что вы будете создавать

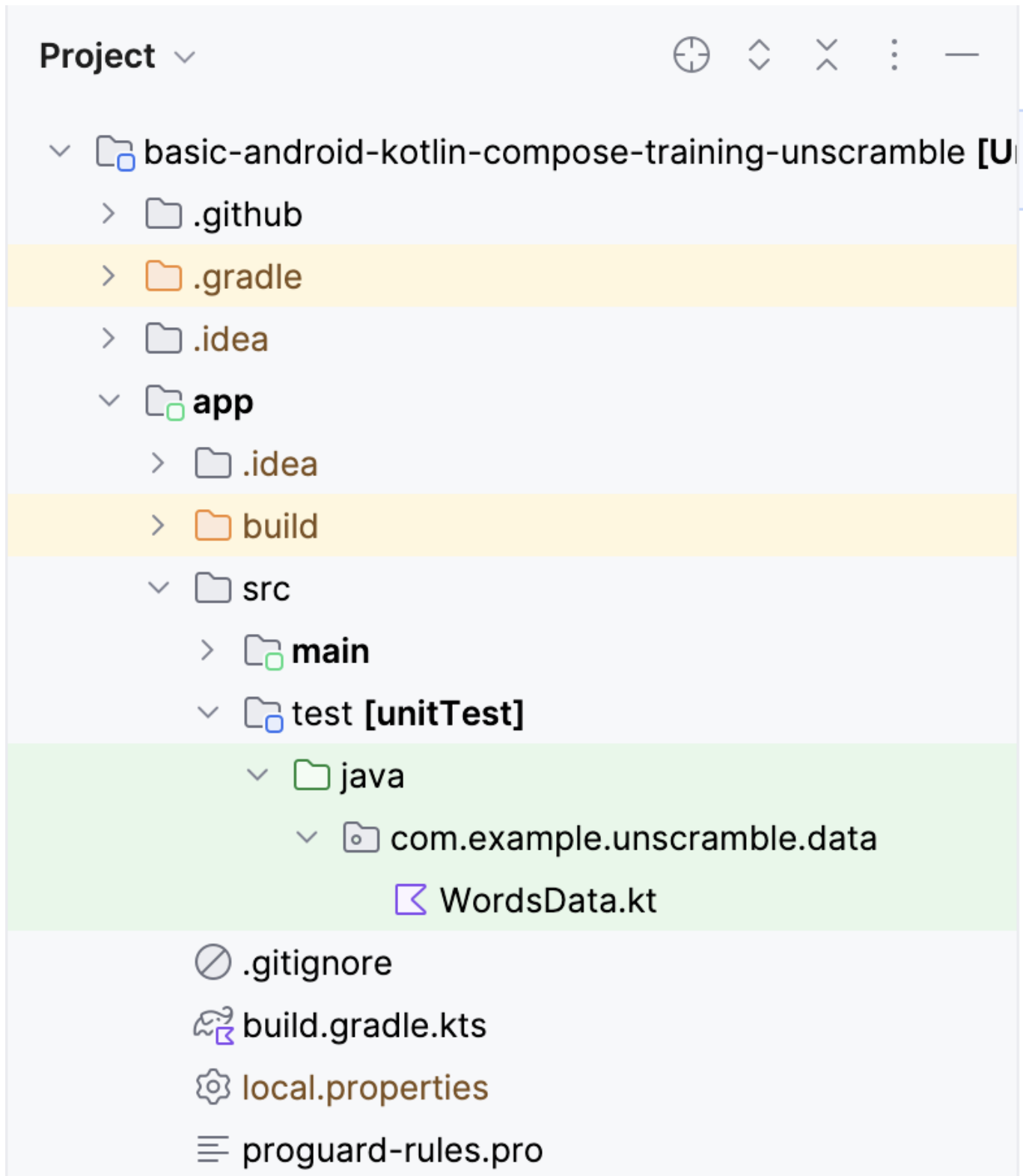
- Модульные тесты для игрового приложения Unscramble

Что вам понадобится

- Последняя версия Android Studio

Обзор стартового кода

В Главе 2 вы научились размещать код модульных тестов в наборе исходных текстов тестов, который находится в папке `src`, как показано на следующем изображении:



Стартовый код содержит следующий файл:

WordsData.kt: Этот файл содержит список слов для тестирования и вспомогательную функцию `getUnscrambledWord()` для получения нешифрованного слова из зашифрованного. Вам не нужно изменять этот файл.

Примечание: Свойство `allWords` в файле `WordsData.kt` из набора тестовых исходников будет использоваться моделью `GameViewModel`. При выполнении тестов оно заменяет свойство `allWords`, доступное в исходном наборе `WordsData.kt` приложения.

В следующем разделе вы узнаете о новой технике, называемой Dependency Injection, которая способствует свободному соединению и помогает использовать различные ресурсы при выполнении тестов.

3. Добавление тестовых зависимостей

В этом коделабе вы используете фреймворк JUnit для написания модульных тестов. Чтобы использовать фреймворк, вам нужно добавить его в качестве зависимости в файл `build.gradle.kts` вашего модуля приложения.

Вы используете конфигурацию реализации для указания зависимостей, которые требуются вашему приложению. Например, чтобы использовать библиотеку `ViewModel` в своем приложении, вы должны добавить зависимость `androidx.lifecycle:lifecycle-viewmodel-compose`, как показано в следующем фрагменте кода:

```
dependencies {  
    ...  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.6.1")  
}
```

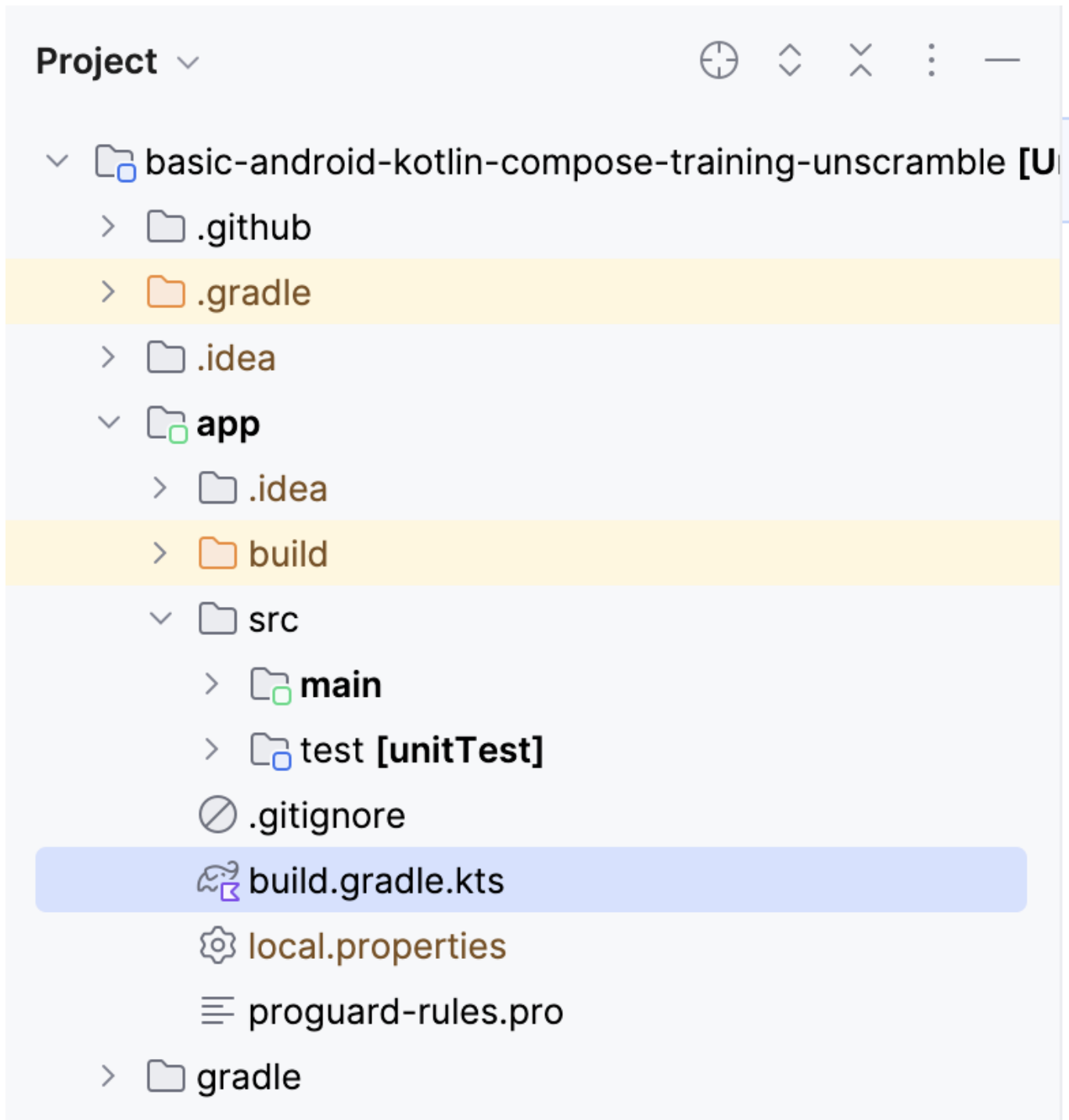
Теперь вы можете использовать эту библиотеку в исходном коде своего приложения, а Android studio поможет добавить ее в создаваемый файл пакета приложений (APK). Однако вы не хотите, чтобы тестовый код был частью вашего APK-файла. Тестовый код не добавляет никакой функциональности, которую бы использовал пользователь, и к тому же он влияет на размер APK. То же самое касается зависимостей, необходимых для тестового кода; их следует держать отдельно. Для этого используется конфигурация `testImplementation`, которая указывает, что конфигурация применяется к локальному исходному коду теста, а не к коду приложения.

Примечание: Каждое приложение Android компилируется и упаковывается в один файл, называемый файлом пакета приложения (APK), который включает в себя весь код приложения, ресурсы, активы и файл манифеста. Для удобства файл пакета приложений часто называют APK, и он имеет расширение `.apk`. Устройства на базе Android используют этот файл для установки приложения.

Чтобы добавить зависимость в проект, укажите конфигурацию зависимости (например, `implementation` или `testImplementation`) в блоке `dependencies` файла `build.gradle.kts`. Каждая конфигурация зависимости предоставляет Gradle различные инструкции о том, как использовать зависимость.

Чтобы добавить зависимость:

Откройте файл `build.gradle.kts` модуля `app`, расположенный в каталоге `app` на панели Project.

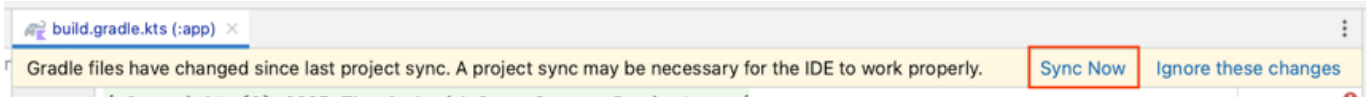


Внутри файла прокрутите вниз, пока не найдете блок `dependencies{}`. Добавьте зависимость, используя конфигурацию `testImplementation` для `junit`.

```
plugins {  
    ...  
}  
  
android {  
    ...  
}  
  
dependencies {  
    ...  
}
```

```
testImplementation("junit:junit:4.13.2")
}
```

В панели уведомлений в верхней части файла build.gradle.kts нажмите Sync Now, чтобы импорт и сборка завершились, как показано на следующем снимке экрана:



Compose Bill of Materials (BOM) Compose BOM - это рекомендуемый способ управления версиями библиотек Compose. BOM позволяет управлять всеми версиями библиотек Compose, указывая только версию BOM.

Обратите внимание на раздел зависимостей в файле build.gradle.kts модуля приложения.

```
// No need to copy over
// This is part of starter code
dependencies {

    // Import the Compose BOM
    implementation (platform("androidx.compose.compose-bom:2023.06.01"))
    ...
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-graphics")
    implementation("androidx.compose.ui:ui-tooling-preview")
    ...
}
```

Обратите внимание на следующее:

Номера версий библиотек Compose не указаны. BOM импортируется с помощью реализации platform(«androidx.compose.compose-bom:2023.06.01»). Это связано с тем, что сам BOM содержит ссылки на последние стабильные версии различных библиотек Compose, таким образом, чтобы они хорошо работали вместе. При использовании BOM в вашем приложении вам не нужно добавлять версию к зависимостям библиотек Compose. Когда вы обновляете версию BOM, все используемые вами библиотеки автоматически обновляются до новых версий.

Чтобы использовать BOM с библиотеками тестирования Compose (инструментальные тесты), вам нужно импортировать androidTestImplementation platform(«androidx.compose.compose-bom:xxxx.xx.xx»). Вы можете создать переменную и повторно использовать ее для реализации и androidTestImplementation, как показано на рисунке.

```
// Example, not need to copy over
dependencies {

    // Import the Compose BOM
```

```
implementation(platform("androidx.compose:compose-bom:2023.06.01"))
implementation("androidx.compose.material:material")
implementation("androidx.compose.ui:ui")
implementation("androidx.compose.ui:ui-tooling-preview")

// ...
androidTestImplementation(platform("androidx.compose:compose-bom:2023.06.01"))
androidTestImplementation("androidx.compose.ui:ui-test-junit4")

}
```

Примечание: Compose BOM предназначен только для библиотек Compose, но не для других библиотек, таких как библиотека `lifecycle androidx.lifecycle`.

Отлично! Вы успешно добавили тестовые зависимости в приложение и узнали о BOM. Теперь вы готовы добавить несколько модульных тестов.

4. Стратегия тестирования

Хорошая стратегия тестирования вращается вокруг покрытия различных путей и границ вашего кода. На самом базовом уровне вы можете разделить тесты на три сценария: путь успеха, путь ошибки и пограничный случай.

Путь успеха: Тесты пути успеха - также известные как тесты счастливого пути - сосредоточены на тестировании функциональности для положительного потока. Положительный поток - это поток, который не имеет исключений или условий ошибки. По сравнению со сценариями пути ошибок и граничного случая, для путей успеха легко создать исчерпывающий список сценариев, поскольку они сосредоточены на предполагаемом поведении вашего приложения. Примером успешного пути в приложении `Unscramble` является корректное обновление счета, количества слов и зашифрованного слова, когда пользователь вводит правильное слово и нажимает кнопку `Submit`.

Путь ошибок: Тесты пути ошибок направлены на проверку функциональности для негативного потока - то есть на проверку того, как приложение реагирует на условия ошибки или недействительный пользовательский ввод. Определить все возможные потоки ошибок довольно сложно, потому что существует множество возможных результатов, когда запланированное поведение не достигается. Один из общих советов - перечислить все возможные пути ошибок, написать для них тесты и продолжать развивать свои модульные тесты по мере обнаружения различных сценариев.

Пример пути ошибки в приложении `Unscramble`: пользователь вводит неправильное слово и нажимает на кнопку `Submit`, в результате чего появляется сообщение об ошибке, а счет и количество слов не обновляются.

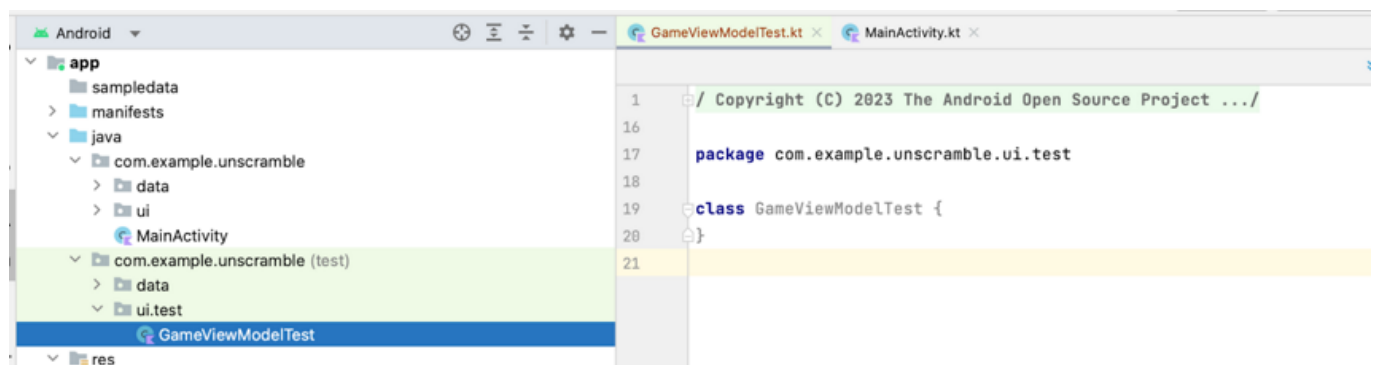
Граничный случай: Граничный случай направлен на тестирование граничных условий в приложении. В приложении `Unscramble` границами являются проверка состояния пользовательского интерфейса при загрузке приложения и состояние пользовательского интерфейса после того, как пользователь сыграет максимальное количество слов. Создание тестовых сценариев по этим категориям может служить руководством для вашего плана тестирования.

Создание тестов Хороший юнит-тест обычно обладает следующими четырьмя свойствами:

Целенаправленный: Она должна быть направлена на тестирование единицы, например фрагмента кода. Этим куском кода часто является класс или метод. Тест должен быть узким и направленным на проверку корректности отдельных частей кода, а не нескольких частей кода одновременно. **Понятный:** Код должен быть простым и понятным при чтении. С первого взгляда разработчик должен быть в состоянии сразу понять замысел, лежащий в основе теста. **Детерминированность:** тест должен последовательно проходить или не проходить. При выполнении тестов любое количество раз, без внесения изменений в код, тест должен давать один и тот же результат. Тест не должен быть нестабильным, когда в одном случае он проваливается, а в другом - проходит, несмотря на отсутствие изменений в коде. **Самодостаточность:** Не требует взаимодействия с человеком или настройки и выполняется изолированно. **Путь к успеху** Чтобы написать юнит-тест для успешного пути, нужно утверждать, что, учитывая, что экземпляр `GameViewModel` был инициализирован, когда вызывается метод `updateUserGuess()` с правильным словом-угадкой, за которым следует вызов метода `checkUserGuess()`, то:

Правильная отгадка передается в метод `updateUserGuess()`. Вызывается метод `checkUserGuess()`. Значение оценки и статуса `isGuessedWordWrong` обновляются корректно. Выполните следующие шаги для создания теста:

Создайте новый пакет `com.example.android.unscramble.ui.test` в наборе исходных текстов `test` и добавьте файл, как показано на следующем снимке экрана:



Чтобы написать модульный тест для класса `GameViewModel`, вам нужен экземпляр класса, чтобы вы могли вызывать его методы и проверять состояние.

В теле класса `GameViewModelTest` объявите свойство `viewModel` и присвойте ему экземпляр класса `GameViewModel`.

```
class GameViewModelTest {
    private val viewModel = GameViewModel()
}
```

Чтобы написать модульный тест для пути успеха, создайте функцию `gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset()` и аннотируйте ее аннотацией `@Test`.

```
class GameViewModelTest {
    private val viewModel = GameViewModel()

    @Test
    fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
    }
}
```

Импортируйте следующее:

```
import org.junit.Test
```

Примечание: В приведенном выше коде для названия тестовой функции используется формат `thingUnderTest_TriggerOfTest_ResultOfTest`:

`thingUnderTest = gameViewModel` `TriggerOfTest = CorrectWordGuessed` `ResultOfTest = ScoreUpdatedAndErrorFlagUnset`

Чтобы передать правильное слово игрока в метод `viewModel.updateUserGuess()`, необходимо получить правильное незашифрованное слово из зашифрованного слова в `GameUiState`. Для этого сначала получите текущее состояние игрового интерфейса.

В теле функции создайте переменную `currentGameUiState` и присвойте ей значение `viewModel.uiState.value`.

```
@Test
fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
    var currentGameUiState = viewModel.uiState.value
}
```

Предупреждение: Этот способ получения `uiState` работает, потому что вы использовали `MutableStateFlow`. В следующих разделах вы узнаете о расширенных вариантах использования `StateFlow`, которые создают поток данных, и вам нужно реагировать на обработку этого потока. Для этих сценариев вы будете писать модульные тесты, используя различные методы и подходы.

Чтобы получить правильную отгадку игрока, используйте функцию `getUnscrambledWord()`, которая принимает в качестве аргумента `currentGameUiState.currentScrambledWord` и возвращает нешифрованное слово. Сохраните это возвращаемое значение в новой переменной с именем `correctPlayerWord`, доступной только для чтения, и присвойте ей значение, возвращаемое функцией `getUnscrambledWord()`.

```
@Test
fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
    var currentGameUiState = viewModel.uiState.value
    val correctPlayerWord =
```

```
getUnscrambledWord(currentGameState.currentScrambledWord)

}
```

Чтобы проверить, правильно ли угадано слово, добавьте вызов метода `viewModel.updateUserGuess()` и передайте в качестве аргумента переменную `correctPlayerWord`. Затем добавьте вызов метода `viewModel.checkUserGuess()` для проверки угаданного слова.

```
@Test
fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
    var currentGameState = viewModel.uiState.value
    val correctPlayerWord =
        getUnscrambledWord(currentGameState.currentScrambledWord)

    viewModel.updateUserGuess(correctPlayerWord)
    viewModel.checkUserGuess()
}
```

Теперь вы готовы утверждать, что состояние игры соответствует вашим ожиданиям.

Получите экземпляр класса `GameState` из значения свойства `viewModel.uiState` и сохраните его в переменной `currentGameState`.

```
@Test
fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
    var currentGameState = viewModel.uiState.value
    val correctPlayerWord =
        getUnscrambledWord(currentGameState.currentScrambledWord)
    viewModel.updateUserGuess(correctPlayerWord)
    viewModel.checkUserGuess()

    currentGameState = viewModel.uiState.value
}
```

Now you are ready to assert that the game state matches your expectations.

Get an instance of the `GameState` class from the value of the `viewModel.uiState` property and store it in the `currentGameState` variable.

```
@Test
fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
    var currentGameState = viewModel.uiState.value
    val correctPlayerWord =
        getUnscrambledWord(currentGameState.currentScrambledWord)
    viewModel.updateUserGuess(correctPlayerWord)
    viewModel.checkUserGuess()
```

```
currentGameUiState = viewModel.uiState.value
// Assert that checkUserGuess() method updates isGuessedWordWrong is updated
correctly.
assertFalse(currentGameUiState.isGuessedWordWrong)
// Assert that score is updated correctly.
assertEquals(20, currentGameUiState.score)
}
```

Импортируйте следующее:

```
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
```

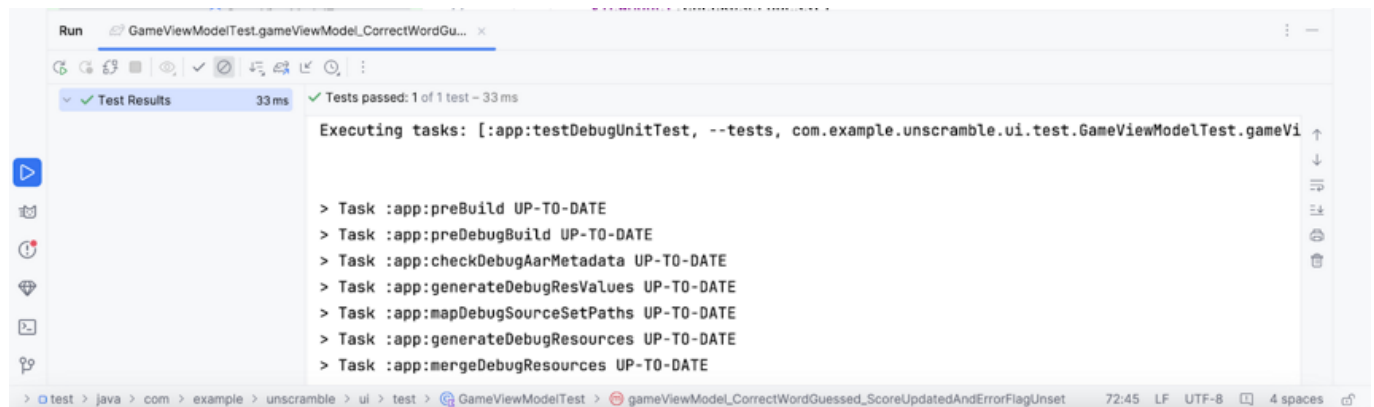
Чтобы сделать значение 20 читаемым и многократно используемым, создайте объект-компаньон и присвойте 20 частной константе с именем SCORE_AFTER_FIRST_CORRECT_ANSWER. Обновите тест с помощью вновь созданной константы.

```
class GameViewModelTest {
    ...
    @Test
    fun gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset() {
        ...
        // Assert that score is updated correctly.
        assertEquals(SCORE_AFTER_FIRST_CORRECT_ANSWER, currentGameUiState.score)
    }

    companion object {
        private const val SCORE_AFTER_FIRST_CORRECT_ANSWER = SCORE_INCREASE
    }
}
```

Запустите тест.

Тест должен пройти, поскольку все утверждения были верны, как показано на следующем снимке экрана:



Путь ошибок Чтобы написать модульный тест для пути ошибок, нужно утверждать, что если в качестве аргумента методу `viewModel.updateUserGuess()` передано неверное слово и вызван метод `viewModel.checkUserGuess()`, то происходит следующее:

Значение свойства `currentGameUiState.score` остается неизменным. Значение свойства `currentGameUiState.isGuessedWordWrong` устанавливается в `true`, поскольку отгадка неверна. Выполните следующие шаги для создания теста:

В теле класса `GameViewModelTest` создайте функцию `gameViewModel_IncorrectGuess_ErrorFlagSet()` и аннотируйте ее аннотацией `@Test`.

```
@Test
fun gameViewModel_IncorrectGuess_ErrorFlagSet() {

}
```

Определите переменную `incorrectPlayerWord` и присвойте ей значение «и», которого не должно быть в списке слов.

```
@Test
fun gameViewModel_IncorrectGuess_ErrorFlagSet() {
    // Given an incorrect word as input
    val incorrectPlayerWord = "and"
}
```

Добавьте вызов метода `viewModel.updateUserGuess()` и передайте в качестве аргумента переменную `incorrectPlayerWord`. Добавьте вызов метода `viewModel.checkUserGuess()` для проверки угадывания.

```
@Test
fun gameViewModel_IncorrectGuess_ErrorFlagSet() {
    // Given an incorrect word as input
    val incorrectPlayerWord = "and"

    viewModel.updateUserGuess(incorrectPlayerWord)
    viewModel.checkUserGuess()
}
```

Добавьте переменную `currentGameUiState` и присвойте ей значение состояния `viewModel.uiState.value`. Используйте функции `assertion`, чтобы утверждать, что значение свойства `currentGameUiState.score` равно 0, а значение свойства `currentGameUiState.isGuessedWordWrong` установлено в `true`.

```
@Test
fun gameViewModel_IncorrectGuess_ErrorFlagSet() {
    // Given an incorrect word as input
    val incorrectPlayerWord = "and"
```

```
viewModel.updateUserGuess(incorrectPlayerWord)
viewModel.checkUserGuess()

val currentGameUiState = viewModel.uiState.value
// Assert that score is unchanged
assertEquals(0, currentGameUiState.score)
// Assert that checkUserGuess() method updates isGuessedWordWrong correctly
assertTrue(currentGameUiState.isGuessedWordWrong)
}
```

Импортируйте следующее:

```
import org.junit.Assert.assertTrue
```

Запустите тест, чтобы убедиться в его прохождении. Граничный случай Чтобы проверить начальное состояние пользовательского интерфейса, необходимо написать модульный тест для класса GameViewModel. Тест должен утверждать, что при инициализации GameViewModel верно следующее:

свойство CurrentWordCount установлено в 1. свойство score установлено в 0. Свойство isGuessedWordWrong имеет значение false. Свойство isGameOver имеет значение false. Выполните следующие шаги, чтобы добавить тест:

Создайте метод gameViewModel_Initialization_FirstWordLoaded() и аннотируйте его с помощью аннотации @Test.

```
@Test
fun gameViewModel_Initialization_FirstWordLoaded() {

}
```

Обратитесь к свойству viewModel.uiState.value, чтобы получить начальный экземпляр класса GameUiState. Присвойте его новой переменной gameUiState, доступной только для чтения.

```
@Test
fun gameViewModel_Initialization_FirstWordLoaded() {
    val gameUiState = viewModel.uiState.value
}
```

Чтобы получить правильное слово игрока, используйте функцию getUnscrambledWord(), которая принимает слово gameUiState.currentScrambledWord и возвращает нешифрованное слово. Присвойте возвращаемое значение новой переменной с именем unScrambledWord, доступной только для чтения.

```
@Test
fun gameViewModel_Initialization_FirstWordLoaded() {
    val gameUiState = viewModel.uiState.value
    val unScrambledWord = getUnscrambledWord(gameUiState.currentScrambledWord)

}
```

Чтобы убедиться в правильности состояния, добавьте функции `assertTrue()` для подтверждения того, что свойство `currentWordCount` установлено в 1, а свойство `score` - в 0. Добавьте функции `assertFalse()`, чтобы убедиться, что свойство `isGuessedWordWrong` равно `false`, а свойство `isGameOver` установлено в `false`.

```
@Test
fun gameViewModel_Initialization_FirstWordLoaded() {
    val gameUiState = viewModel.uiState.value
    val unScrambledWord = getUnscrambledWord(gameUiState.currentScrambledWord)

    // Assert that current word is scrambled.
    assertEquals(unScrambledWord, gameUiState.currentScrambledWord)
    // Assert that current word count is set to 1.
    assertTrue(gameUiState.currentWordCount == 1)
    // Assert that initially the score is 0.
    assertTrue(gameUiState.score == 0)
    // Assert that the wrong word guessed is false.
    assertFalse(gameUiState.isGuessedWordWrong)
    // Assert that game is not over.
    assertFalse(gameUiState.isGameOver)
}
```

Импортируйте следующее:

```
import org.junit.Assert.assertEquals
```

Запустите тест, чтобы убедиться в его прохождении. Другой пограничный случай - проверка состояния пользовательского интерфейса после того, как пользователь угадал все слова. Вам нужно утверждать, что если пользователь правильно угадал все слова, то верно следующее:

Счет является актуальным; Свойство `currentGameUiState.currentWordCount` равно значению константы `MAX_NO_OF_WORDS`; Свойство `currentGameUiState.isGameOver` имеет значение `true`. Выполните следующие шаги, чтобы добавить тест:

Создайте метод `gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly()` и аннотируйте его аннотацией `@Test`. В методе создайте переменную `expectedScore` и присвойте ей значение 0.

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
}
```

Чтобы получить начальное состояние, добавьте переменную `currentGameUiState` и присвойте ей значение свойства `viewModel.uiState.value`.

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
    var currentGameUiState = viewModel.uiState.value
}
```

Чтобы получить правильное слово игрока, используйте функцию `getUnscrambledWord()`, которая принимает слово `currentGameUiState.currentScrambledWord` и возвращает нешифрованное слово. Сохраните это возвращаемое значение в новой переменной с именем `correctPlayerWord`, доступной только для чтения, и присвойте ей значение, возвращаемое функцией `getUnscrambledWord()`.

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
    var currentGameUiState = viewModel.uiState.value
    var correctPlayerWord =
        getUnscrambledWord(currentGameUiState.currentScrambledWord)
}
```

Чтобы проверить, угадал ли пользователь все ответы, используйте блок `repeat`, чтобы повторить выполнение метода `viewModel.updateUserGuess()` и метода `viewModel.checkUserGuess()` `MAX_NO_OF_WORDS` несколько раз.

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
    var currentGameUiState = viewModel.uiState.value
    var correctPlayerWord =
        getUnscrambledWord(currentGameUiState.currentScrambledWord)
    repeat(MAX_NO_OF_WORDS) {

    }
}
```

В блоке `repeat` добавьте значение константы `SCORE_INCREASE` в переменную `expectedScore`, чтобы утверждать, что оценка увеличивается после каждого правильного ответа. Добавьте вызов метода

`viewModel.updateUserGuess()` и передайте в качестве аргумента переменную `correctPlayerWord`. Добавьте вызов метода `viewModel.checkUserGuess()`, чтобы запустить проверку пользовательской догадки.

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
    var currentGameUiState = viewModel.uiState.value
    var correctPlayerWord =
        getUnscrambledWord(currentGameUiState.currentScrambledWord)
    repeat(MAX_NO_OF_WORDS) {
        expectedScore += SCORE_INCREASE
        viewModel.updateUserGuess(correctPlayerWord)
        viewModel.checkUserGuess()
    }
}
```

Для обновления текущего слова игрока используйте функцию `getUnscrambledWord()`, которая принимает в качестве аргумента `currentGameUiState.currentScrambledWord` и возвращает нешифрованное слово. Сохраните возвращаемое значение в новой переменной с именем `correctPlayerWord`, доступной только для чтения. Чтобы убедиться в правильности состояния, добавьте функцию `assertEquals()` для проверки того, что значение свойства `currentGameUiState.score` равно значению переменной `expectedScore`.

```
@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
    var currentGameUiState = viewModel.uiState.value
    var correctPlayerWord =
        getUnscrambledWord(currentGameUiState.currentScrambledWord)
    repeat(MAX_NO_OF_WORDS) {
        expectedScore += SCORE_INCREASE
        viewModel.updateUserGuess(correctPlayerWord)
        viewModel.checkUserGuess()
        currentGameUiState = viewModel.uiState.value
        correctPlayerWord =
            getUnscrambledWord(currentGameUiState.currentScrambledWord)
        // Assert that after each correct answer, score is updated correctly.
        assertEquals(expectedScore, currentGameUiState.score)
    }
}
```

Добавьте функцию `assertEquals()` для проверки того, что значение свойства `currentGameUiState.currentWordCount` равно значению константы `MAX_NO_OF_WORDS` и что значение свойства `currentGameUiState.isGameOver` установлено в `true`.

```

@Test
fun gameViewModel_AllWordsGuessed_UiStateUpdatedCorrectly() {
    var expectedScore = 0
    var currentGameUiState = viewModel.uiState.value
    var correctPlayerWord =
        getUnscrambledWord(currentGameUiState.currentScrambledWord)
    repeat(MAX_NO_OF_WORDS) {
        expectedScore += SCORE_INCREASE
        viewModel.updateUserGuess(correctPlayerWord)
        viewModel.checkUserGuess()
        currentGameUiState = viewModel.uiState.value
        correctPlayerWord =
            getUnscrambledWord(currentGameUiState.currentScrambledWord)
        // Assert that after each correct answer, score is updated correctly.
        assertEquals(expectedScore, currentGameUiState.score)
    }
    // Утверждаем, что после того, как все вопросы будут отвечены, текущее
    // количество слов будет актуальным.
    assertEquals(MAX_NO_OF_WORDS, currentGameUiState.currentWordCount)
    // Утверждаем, что после ответа на 10 вопросов игра заканчивается.
    assertTrue(currentGameUiState.isGameOver)
}

```

Import the following:

```
import com.example.unscramble.data.MAX_NO_OF_WORDS
```

Запустите тест, чтобы убедиться в его прохождении. Обзор жизненного цикла тестового экземпляра. При внимательном рассмотрении того, как `viewModel` инициализируется в тесте, можно заметить, что `viewModel` инициализируется только один раз, хотя все тесты используют его. В этом фрагменте кода показано определение свойства `viewModel`.

```

class GameViewModelTest {
    private val viewModel = GameViewModel()

    @Test
    fun gameViewModel_Initialization_FirstWordLoaded() {
        val gameUiState = viewModel.uiState.value
        ...
    }
    ...
}

```

Вы можете задаться следующими вопросами:

Означает ли это, что один и тот же экземпляр `viewModel` будет повторно использоваться во всех тестах? Вызовет ли это какие-либо проблемы? Например, что если тестовый метод `gameViewModel_Initialization_FirstWordLoaded` будет выполняться после тестового метода `gameViewModel_CorrectWordGuessed_ScoreUpdatedAndErrorFlagUnset`? Провалится ли тест инициализации? Ответ на оба вопроса - нет. Тестовые методы выполняются изолированно, чтобы избежать неожиданных побочных эффектов от изменяемого состояния тестового экземпляра. По умолчанию перед выполнением каждого тестового метода JUnit создает новый экземпляр тестового класса.

Поскольку в вашем классе `GameViewModelTest` на данный момент четыре тестовых метода, `GameViewModelTest` инстанцируется четыре раза. У каждого экземпляра есть своя копия свойства `viewModel`. Следовательно, последовательность выполнения теста не имеет значения.

Примечание: Такой жизненный цикл экземпляра теста «для каждого метода» является поведением по умолчанию, начиная с JUnit4.

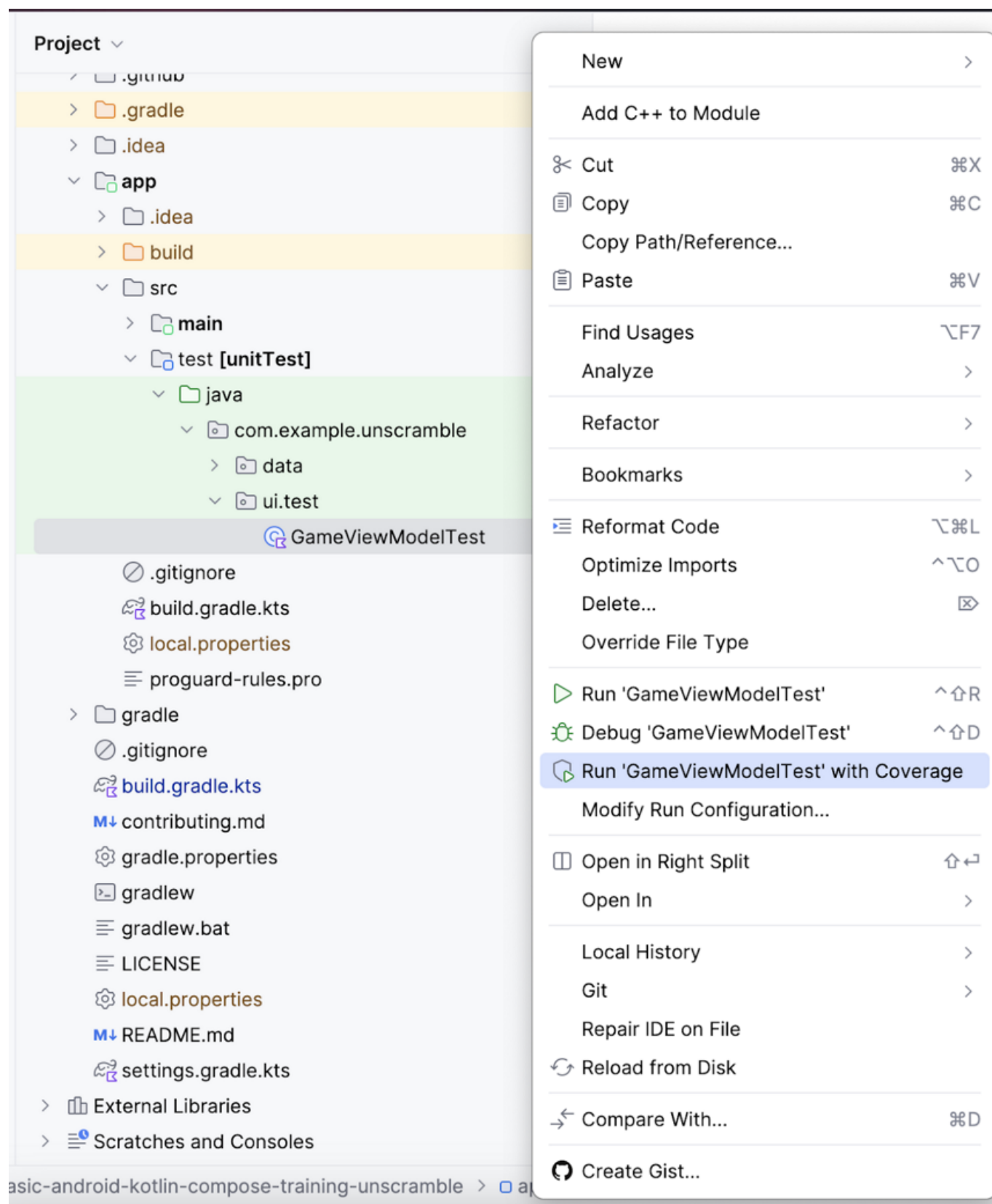
5. Введение в покрытие кода

Покрытие кода играет важную роль в определении того, насколько адекватно вы тестируете классы, методы и строки кода, из которых состоит ваше приложение.

Android Studio предоставляет инструмент покрытия тестов для локальных юнит-тестов, позволяющий отслеживать процент и области кода вашего приложения, которые покрыли ваши юнит-тесты.

Запуск тестов с покрытием с помощью Android Studio Чтобы запустить тесты с покрытием:

Щелкните правой кнопкой мыши файл `GameViewModelTest.kt` на панели проекта и выберите `cf4c5adfe69a119f.png` Run 'GameViewModelTest' with Coverage.



После завершения выполнения теста на панели покрытия справа выберите опцию Flatten Packages.

CoverageGameViewModelTest

Flatten Packages

Element	Class, %	Method, %	Line, %
example	13% (3/23)	11% (11/98)	17% (59/334)

Обратите внимание на пакет `com.example.android.unscramble.ui`, как показано на следующем изображении.

CoverageGameViewModelTest

Element	Class, %	Method, %	Line, %
all			
com.example.unscramble	0% (0/1)	0% (0/2)	0% (0/4)
com.example.unscramble.data	100% (1/1)	100% (3/3)	100% (14/14)
com.example.unscramble.ui	11% (2/17)	27% (8/29)	26% (45/172)
com.example.unscramble.ui.theme	0% (0/4)	0% (0/64)	0% (0/144)

Дважды щелкните на имени пакета `com.example.android.unscramble.ui`, и на экране появится покрытие для `GameViewModel`, как показано на следующем изображении:

CoverageGameViewModelTest

Element	Class, %	Method, %	Line, %
all			
com.example.unscramble	0% (0/1)	0% (0/2)	0% (0/4)
com.example.unscramble.data	100% (1/1)	100% (3/3)	100% (14/14)
com.example.unscramble.ui	11% (2/17)	27% (8/29)	26% (45/172)
GameScreenKt	0% (0/15)	0% (0/20)	0% (0/125)
GameUiState	100% (1/1)	100% (1/1)	100% (6/6)
GameViewModel	100% (1/1)	87% (7/8)	95% (39/41)
com.example.unscramble.ui.theme	0% (0/4)	0% (0/64)	0% (0/144)

Анализ отчета о тестировании Отчет, показанный на следующей диаграмме, разбит на два аспекта:

Процент методов, покрытых модульными тестами: В приведенном примере тесты, которые вы написали, покрыли 7 из 8 методов. Это 87 % от общего числа методов. Процент строк, покрытых модульными тестами: На диаграмме примера написанные вами тесты покрыли 39 из 41 строки кода. Это 95 % строк кода.

Примечание: Покрытие кода отслеживает количество строк, выполненных во время выполнения тестов. Таким образом, несмотря на отсутствие проверок на GameState, покрытие показывает 100%.

Судя по отчетам, написанные вами модульные тесты пропустили некоторые части кода. Чтобы определить, какие части были пропущены, выполните следующее действие:

Дважды щелкните GameViewModel.

com.example.unscramble.ui	11% (2/17)	27%
GameScreenKt	0% (0/15)	0%
GameState	100% (1/1)	100%
GameViewModel	100% (1/1)	87%
com.example.unscramble.ui.theme	0% (0/4)	0%

Android Studio отображает файл GameViewModel.kt с дополнительным цветовым кодированием в левой части окна. Ярко-зеленый цвет указывает на то, что эти строки кода были закрыты.

```
16
17 package com.example.unscramble.ui
18
19 > import ...
20
21 /**
22  * ViewModel containing the app data and methods to process the data
23  */
24 @Android Dev
25 class GameViewModel : ViewModel() {
26
27     // Game UI state
28     private val _uiState = MutableStateFlow(GameUiState())
29     val uiState: StateFlow<GameUiState> = _uiState.asStateFlow()
30
31     @Android Dev
32     var userGuess by mutableStateOf( value: "" )
33     private set
34
35     // Set of words used in the game
36     private var usedWords: MutableSet<String> = mutableSetOf()
37     private lateinit var currentWord: String
38
39     @Android Dev
40     init {
41         resetGame()
42     }
43 }
```

Прокручивая страницу GameViewModel вниз, вы можете заметить, что несколько строк отмечены светло-розовым цветом. Этот цвет указывает на то, что эти строки кода не были охвачены модульными тестами.

```

86      /*
87      * Skip to next word
88      */
89      fun skipWord() {
90          updateGameState(_uiState.value.score)
91          // Reset user guess
92          updateUserGuess( guessedWord: "" )
93      }

```

Улучшение покрытия Чтобы улучшить покрытие, нужно написать тест, который покроет недостающий путь. Нужно добавить тест, утверждающий, что когда пользователь пропускает слово, то верно следующее:

свойство `currentGameUiState.score` остается неизменным. Свойство `currentGameUiState.currentWordCount` увеличивается на единицу, как показано в следующем фрагменте кода. Чтобы подготовиться к улучшению покрытия, добавьте следующий метод тестирования в класс `GameViewModelTest`.

```

@Test
fun gameViewModel_WordSkipped_ScoreUnchangedAndWordCountIncreased() {
    var currentGameUiState = viewModel.uiState.value
    val correctPlayerWord =
        getUnscrambledWord(currentGameUiState.currentScrambledWord)
    viewModel.updateUserGuess(correctPlayerWord)
    viewModel.checkUserGuess()

    currentGameUiState = viewModel.uiState.value
    val lastWordCount = currentGameUiState.currentWordCount
    viewModel.skipWord()
    currentGameUiState = viewModel.uiState.value
    // Assert that score remains unchanged after word is skipped.
    assertEquals(SCORE_AFTER_FIRST_CORRECT_ANSWER, currentGameUiState.score)
    // Assert that word count is increased by 1 after word is skipped.
    assertEquals(lastWordCount + 1, currentGameUiState.currentWordCount)
}

```

Выполните следующие действия, чтобы повторно запустить покрытие:

Щелкните правой кнопкой мыши файл `GameViewModelTest.kt` и в меню выберите `Run 'GameViewModelTest' with Coverage`. После успешной сборки снова перейдите к элементу `GameViewModel` и убедитесь, что процент покрытия составляет 100 %. Окончательный отчет о покрытии показан на следующем изображении.

com.example.unscramble.ui	11% (2/17)	31% (9/29)	27% (47/172)
GameScreenKt	0% (0/15)	0% (0/20)	0% (0/125)
GameUiState	100% (1/1)	100% (1/1)	100% (6/6)
GameViewModel	100% (1/1)	100% (8/8)	100% (41/41)

Перейдите к файлу `GameViewModel.kt` и прокрутите его вниз, чтобы проверить, пройден ли ранее пропущенный путь.

```

86      /*
87      * Skip to next word
88      */
89      fun skipWord() {
90          updateGameState(_uiState.value.score)
91          // Reset user guess
92          updateUserGuess( guessedWord: "" )
93      }

```

Вы узнали, как выполнять, анализировать и улучшать покрытие кода вашего приложения.

Означает ли высокий процент покрытия кода высокое качество кода приложения? Нет. Покрытие кода указывает на процент кода, покрытого или выполненного вашим модульным тестом. Это не означает, что код проверен. Если вы удалите все утверждения из кода вашего модульного теста и запустите покрытие кода, оно все равно покажет 100% покрытие.

Высокое покрытие не означает, что тесты разработаны правильно и что тесты проверяют поведение приложения. Вам нужно убедиться, что тесты, которые вы написали, содержат утверждения, которые проверяют поведение тестируемого класса. Также не нужно стремиться писать юнит-тесты, чтобы получить 100-процентное покрытие тестами всего приложения. Некоторые части кода приложения, такие как `Activities`, следует тестировать с помощью UI-тестов.

Однако низкое покрытие означает, что большая часть вашего кода осталась не протестированной. Используйте покрытие кода как инструмент для поиска частей кода, которые не были выполнены вашими тестами, а не как инструмент для измерения качества вашего кода.

Заключение

Вы научились определять стратегию тестирования и реализовали модульные тесты для проверки `ViewModel` и `StateFlow` в приложении `Unscramble`. Продолжая создавать приложения для Android, убедитесь, что вы пишете тесты вместе с функциями ваших приложений, чтобы убедиться, что ваши приложения работают должным образом на протяжении всего процесса разработки.

Резюме

- Используйте конфигурацию `testImplementation`, чтобы указать, что зависимости относятся к локальному исходному коду тестов, а не к коду приложения.
- Стремитесь классифицировать тесты по трем сценариям: Путь к успеху, Путь к ошибке и Пограничный случай.
- Хорошие модульные тесты обладают как минимум четырьмя характеристиками: они сфокусированы, понятны, детерминированы и самодостаточны.
- Тестовые методы выполняются изолированно, чтобы избежать неожиданных побочных эффектов от изменяемого состояния тестового экземпляра.
- По умолчанию перед выполнением каждого тестового метода JUnit создает новый экземпляр тестового класса.
- Покрывание кода играет важную роль в определении того, насколько адекватно вы протестировали классы, методы и строки кода, составляющие ваше приложение.